

Aula 12 - Controle de versão: SVN e Git

TJSC 2026 - Analista de Sistemas - Engenharia de Software e Desenvolvimento - Banca FGV

Material autoral de estudo, calibrado para a banca FGV, com base no edital retificado do TJSC 2026, no Manual Mestre de Calibração FGV para Aulas e Questões e em referências técnicas oficiais de Git e Apache Subversion. A aula não copia questões reais: usa provas oficiais apenas para captar estrutura, vocabulário, densidade e estilo de cobrança.

Item	Definição nesta aula
Aula da trilha	Aula 12 - Controle de versão: SVN e Git.
Tópico do edital	1.15 Ferramentas de controle de versão SVN e Git.
Recorte	Conceitos de controle de versão, arquitetura centralizada/distribuída, comandos essenciais, branching, tags, merge, rebase, conflitos, fluxo com remotos, diferenças SVN x Git e pegadinhas FGV.
Fora do recorte	CI/CD em profundidade fica para a Aula 14; GitHub/GitLab como plataformas são tratados apenas quando ajudam a entender Git.

1. Como a FGV tende a cobrar este assunto

Controle de versão é um assunto pequeno no edital, mas com alta chance de cobrança conceitual por comparação. A FGV costuma explorar enunciados de cenário: um analista compara Git e SVN; uma equipe confunde commit com push; um desenvolvedor executa fetch esperando atualizar a árvore de trabalho; ou um time precisa decidir entre merge e rebase. O ponto de prova raramente é decorar todos os comandos: é distinguir arquitetura, efeito e consequência.

Em prova, fique atento a verbos como "publicar", "sincronizar", "registrar localmente", "travar arquivo", "reescrever histórico", "obter alterações remotas", "incorporar alterações" e "resolver conflito". Eles apontam para comandos e conceitos diferentes.

Formato FGV provável	Como aparece em controle de versão	Risco para o candidato
Comparação entre ferramentas	Git x SVN quanto a arquitetura, repositório local, working copy, file locking, branches e tags.	Marcar alternativa que mistura conceito verdadeiro de uma ferramenta na outra.
Cenário corporativo	Equipe usa feature branch, merge request, branch protegida, conflito em arquivo de configuração.	Responder pela prática do trabalho e não pela definição formal.
Comando e efeito	git fetch, git pull, git commit, git push, svn update, svn commit.	Confundir baixar alterações com incorporá-las; confundir commit local com publicação remota.
Exceção/incorreta	Alternativa diz que Git depende sempre de servidor central ou que SVN é distribuído.	Cair em palavra absoluta: sempre, necessariamente, automaticamente.
V/F e afirmativas	Itens sobre merge/rebase, branch/tag, reset/revert, trunk/branches/tags.	Não perceber que um item é verdadeiro em Git, mas falso em SVN, ou vice-versa.

2. Conceitos fundamentais de controle de versão

Sistema de controle de versão, ou **VCS - Version Control System**, é uma ferramenta que registra alterações em arquivos ao longo do tempo, permitindo recuperar versões anteriores, comparar

mudanças, identificar autores, coordenar trabalho em equipe e manter rastreabilidade. No desenvolvimento de software, versionar não significa apenas "salvar backup": significa construir uma linha histórica auditável do projeto.

Para o TJSC, pense em um sistema corporativo com backend Java/Quarkus, frontend Angular/TypeScript, scripts de banco e arquivos de configuração. O VCS permite saber quando uma regra foi alterada, por quem, em qual demanda, em qual branch e com qual justificativa. Em ambiente público ou bancário, isso conversa com governança, auditoria, rastreabilidade e controle de mudança.

Termo	Ideia de prova	Exemplo prático
Repositório	Base que guarda o histórico versionado do projeto.	Projeto Quarkus com src/main/java, pom.xml, migrations e README.
Working tree / working copy	Área de trabalho em que os arquivos são editados.	Seu diretório local no VS Code.
Commit	Registro de uma alteração no histórico. Em Git, nasce local; em SVN, é enviado ao repositório central.	"Ajusta validação de CPF no cadastro".
Revision	Número ou identificação de versão. Em SVN, a revisão é global no repositório; em Git, cada commit tem hash.	SVN r1540; Git 7f3a9c1...
Branch	Linha paralela de desenvolvimento.	feature/cadastro-processo ou hotfix/corrige-login.
Tag	Marcador de uma versão específica, em geral para release.	v1.4.0 ou release-2026.06.
Merge	Integra alterações de uma linha de desenvolvimento em outra.	Levar feature/cadastro para develop.
Conflito	Situação em que a ferramenta não consegue combinar automaticamente alterações concorrentes.	Dois devs alteram o mesmo trecho de application.properties.

3. Arquiteturas: centralizada x distribuída

Esta é a distinção mais cobrada. SVN - Apache Subversion - é tradicionalmente um sistema centralizado: há um repositório central e os desenvolvedores trabalham com cópias locais que se sincronizam com esse servidor. Git é distribuído: cada clone tem o histórico completo do projeto, permitindo commits locais, branches locais e várias operações sem conexão permanente ao servidor remoto.

Critério	SVN - Subversion	Git
Modelo	Centralizado.	Distribuído.
Histórico local completo	Em regra, a working copy não é uma cópia completa do histórico do repositório.	Cada clone contém o histórico do repositório.
Commit	Publica alteração no repositório central.	Registra alteração no repositório local. Para publicar, usa-se push.
Trabalho offline	Limitado: é possível editar arquivos, mas operações centrais dependem do servidor.	Amplo: commit, branch, log local e diff funcionam sem rede.
Branch e tag	Normalmente organizadas em pastas convencionais trunk/branches/tags; cópias baratas.	Referências leves para commits; branches são comuns no fluxo diário.

Critério	SVN - Subversion	Git
File locking	Suporte nativo típico e relevante para arquivos difíceis de mesclar.	Não é característica nativa central do Git comum; pode existir via extensões/plataformas, como Git LFS locking.
Pegadinha FGV	Dizer que SVN é distribuído ou que commit é local como no Git.	Dizer que Git precisa sempre de servidor central ou que push é sinônimo de commit.

A banca pode perguntar exatamente por essa comparação: Git possui arquitetura distribuída; SVN possui arquitetura centralizada. Se aparecer "file locking nativo", a tendência é associar ao SVN, não ao Git puro. Cuidado: plataformas como GitLab ou GitHub podem oferecer mecanismos complementares, mas isso não muda a natureza conceitual do Git.

4. SVN - Subversion: o que saber para prova

SVN - Subversion - organiza o trabalho em torno de um repositório central e uma working copy. A working copy é o diretório local em que o desenvolvedor edita os arquivos; ela mantém metadados de versionamento em diretório .svn e interage com o repositório por comandos como checkout, update e commit.

4.1 Trunk, branches e tags

A convenção clássica do SVN usa três diretórios no repositório: trunk, branches e tags. O trunk representa a linha principal de desenvolvimento. Branches guarda linhas paralelas. Tags guarda marcações de versões. Tecnicamente, no SVN, branch e tag são cópias dentro do repositório; a diferença é de convenção e governança: tag deve ser tratada como imutável para representar uma versão liberada.

Elemento SVN	Função	Armadilha
trunk	Linha principal do desenvolvimento.	Não é sinônimo obrigatório de produção; depende do fluxo.
branches	Linhas paralelas de desenvolvimento ou manutenção.	Não confundir com branch leve do Git; em SVN é estruturado por cópia no repositório.
tags	Marcação de uma versão, release ou marco.	Tecnicamente pode ser alterada, mas por convenção deve ser imutável.
revision	Número global do repositório após commit.	Não é hash de commit como no Git.

4.2 Comandos essenciais de SVN

Comando	Efeito de prova
svn checkout URL	Cria uma working copy local a partir do repositório.
svn update	Atualiza a working copy com alterações vindas do repositório.
svn status	Mostra estado dos arquivos na working copy.
svn diff	Mostra diferenças entre working copy e base/repositório.
svn add	Agenda arquivo novo para versionamento.
svn commit -m "mensagem"	Envia alterações da working copy para o repositório central, criando nova revisão.
svn revert	Descarta alterações locais não commitadas.
svn merge	Aplica alterações de outra linha/revisão à working copy, exigindo commit posterior.

Comando	Efeito de prova
svn copy	Usado para criar branch ou tag no modelo trunk/branches/tags.
svn lock / svn unlock	Reserva/libera arquivo para evitar edição concorrente, útil em binários.

4.3 Conflitos no SVN

No SVN, conflitos geralmente surgem quando o servidor tem uma alteração em arquivo que também foi alterado localmente de forma incompatível. O candidato deve associar o ciclo comum: **update traz mudanças, conflito é marcado, desenvolvedor resolve manualmente, marca como resolvido e só depois faz commit.** A banca pode tentar vender a ideia de que o SVN sempre impede conflito por ser centralizado. Isso é falso: centralização reduz certos cenários, mas não elimina conflito.

5. Git: o que saber para prova

Git é um sistema distribuído de controle de versão. O comando git clone não cria apenas uma pasta de trabalho: cria um repositório local com histórico, referências, objetos e metadados. Isso explica por que o desenvolvedor pode criar commits localmente, consultar histórico e trabalhar em branches sem conexão permanente com o servidor remoto.

5.1 As três áreas principais do Git

A FGV gosta de confundir working tree, staging area e repositório. Entenda como uma esteira de versionamento local: primeiro você edita arquivos; depois seleciona o que entrará no próximo commit; por fim, registra o commit no histórico local.

Área	Também chamada	Papel
Working tree	Diretório de trabalho	Arquivos editáveis no sistema de arquivos.
Index	Staging area	Área intermediária que seleciona o que entrará no próximo commit.
Local repository	Repositório local	Banco de objetos e histórico de commits locais.
Remote repository	origin, upstream etc.	Repositório remoto usado para colaboração e publicação.

```
# Ciclo básico do Git
git status          # verifica arquivos modificados, staged e untracked
git add src/App.ts  # coloca alteração na staging area
git commit -m "Ajuda validação de formulário" # cria commit local
git push origin feature/formulario           # publica no remoto
```

5.2 Comandos essenciais de Git

Comando	Efeito de prova	Pegadinha
git init	Cria repositório Git em um diretório.	Não baixa projeto remoto.
git clone URL	Copia repositório remoto e configura remote padrão.	Não é igual a svn checkout, pois traz histórico Git.
git status	Mostra estado do working tree e index.	Não cria commit.
git add	Move alterações para a staging area.	Não salva no histórico por si só.
git commit	Cria commit local.	Não publica no servidor.
git fetch	Busca referências/objetos do remoto.	Não integra automaticamente na branch atual.
git pull	Fetch + integração, normalmente merge ou rebase conforme configuração.	Pode gerar conflito.

Comando	Efeito de prova	Pegadinha
git push	Envia commits locais para o remoto.	Não é sinônimo de commit.
git branch	Lista/cria/remove branches.	Criar branch não muda automaticamente para ela, dependendo do comando usado.
git switch / checkout	Muda de branch; checkout também tem usos antigos para arquivos.	checkout é comando multifunção e fonte de pegadinha.
git merge	Integra histórico de outra branch preservando a linha de desenvolvimento.	Pode criar commit de merge.
git rebase	Reaplica commits sobre outra base, reescrevendo histórico dos commits reaplicados.	Evitar em histórico público compartilhado sem cuidado.
git revert	Cria novo commit que desfaz efeito de commit anterior.	Não apaga histórico.
git reset	Move ponteiro da branch; pode alterar index/working tree conforme modo.	Pode reescrever histórico local.
git stash	Guarda temporariamente mudanças locais não commitadas.	Não substitui commit.
git tag	Cria marcador para commit, comum em releases.	Tag não é branch.

5.3 Fetch, pull, push e commit

Essa é uma das zonas de maior retorno em prova. No Git, commit é local; push publica; fetch baixa informações do remoto sem incorporar automaticamente à sua branch; pull faz fetch e integra. Portanto, se a questão diz que o desenvolvedor quer apenas verificar o que mudou no remoto sem alterar a árvore de trabalho, a resposta tende a ser git fetch, não git pull.

Necessidade	Comando mais associado
Registrar alteração no histórico local	git commit
Publicar commits locais no repositório remoto	git push
Buscar alterações remotas sem integrar automaticamente	git fetch
Buscar e integrar alterações remotas	git pull
Selecionar alterações para o próximo commit	git add
Comparar diferenças	git diff

5.4 Merge x rebase

Merge e rebase podem levar ao mesmo conteúdo final, mas produzem históricos diferentes. Merge integra as linhas de desenvolvimento preservando a topologia e, em muitos casos, criando um commit de merge. Rebase reaplica commits de uma branch sobre outra base, produzindo histórico mais linear, mas reescrevendo os commits reaplicados. Essa diferença é cobrável porque a FGV gosta de termos como "histórico linear", "reescrever histórico" e "preservar a ramificação".

Critério	Merge	Rebase
Ideia	Une histórias.	Reaplica commits em nova base.
Histórico	Preserva ramificações; pode criar merge commit.	Tende a tornar o histórico linear.

Critério	Merge	Rebase
Risco	Histórico pode ficar menos linear.	Reescrita de histórico pode complicar trabalho compartilhado.
Uso comum	Integrar feature em develop/main.	Atualizar feature local antes de abrir merge request/pull request.
Pegadinha	Não é simplesmente "mais seguro" sempre.	Não deve ser usado de forma descuidada em commits públicos compartilhados.

5.5 Reset x revert x restore

Comando	Efeito resumido	Quando cai em prova
git revert	Cria novo commit que desfaz outro commit.	Quando se quer desfazer mantendo histórico público.
git reset --soft	Move HEAD e preserva alterações no index.	Quando quer refazer commit sem perder alterações.
git reset --mixed	Move HEAD e tira alterações do index, preservando working tree.	Modo padrão do reset.
git reset --hard	Move HEAD e descarta alterações do index e working tree.	Cuidado: perda de alterações locais.
git restore	Restaura arquivos no working tree ou index.	Comando moderno para casos antes feitos com checkout.

5.6 Branch, tag, fork e remote

Branch é linha de desenvolvimento. Tag é marcador de um ponto do histórico. Fork é cópia de um repositório em uma plataforma de hospedagem, muito usada para contribuição sem permissão direta no repositório original. Remote é uma referência local para um repositório remoto, como origin ou upstream. A FGV pode misturar esses termos em alternativas; não aceite "tag evolui como branch" ou "fork é o mesmo que branch".

6. Git e SVN em ambiente corporativo

Na sua realidade com Java/Quarkus, Angular/TypeScript e APIs, controle de versão não é só código-fonte. Também entram arquivos de build, documentação técnica, coleções de testes, scripts de banco, arquivos de infraestrutura, pipelines e contratos de API. A prova, porém, tende a reduzir ao conceito formal: rastrear mudanças, controlar versões, apoiar colaboração e permitir recuperação.

Artefato	Por que versionar	Cuidado
Código Java/Quarkus	Rastrear regra de negócio, endpoints, DTOs, entidades e testes.	Não versionar target/ ou classes compiladas.
Angular/TypeScript	Controlar componentes, services, guards, rotas e package.json.	node_modules normalmente fica fora do Git via .gitignore.
Scripts SQL/Flyway	Evolução de schema e dados de referência.	Evitar editar migration já aplicada sem política clara.
OpenAPI/Swagger	Contrato entre frontend, backend e consumidores.	Mudança incompatível deve ser revisada.
Pipeline YAML	Reprodutibilidade de build, testes e deploy.	Segredos não devem ser commitados.

Artefato	Por que versionar	Cuidado
Documentação técnica	Histórico de decisões e instruções de operação.	Documentação deve acompanhar mudança relevante.

7. Arquivos que normalmente não devem ser versionados

A FGV pode cobrar boas práticas de forma conceitual. Em Git, o arquivo .gitignore define padrões de arquivos que devem ser ignorados pelo controle de versão. A ideia não é esconder alterações importantes; é evitar versionar artefatos gerados, arquivos temporários, dependências baixáveis, credenciais e dados sensíveis.

Normalmente versionar	Normalmente ignorar
src/, pom.xml, build.gradle, package.json, package-lock.json quando adotado, README, scripts de migration, Dockerfile, docker-compose de desenvolvimento sem segredo, arquivos de pipeline sem credenciais.	target/, build/, dist/, node_modules/, .env com segredo, arquivos de log, binários gerados, cache de IDE, chaves privadas, senhas, tokens e dumps sensíveis.

8. Camadas práticas do Git que viram pegadinha

8.1 HEAD, branch atual e ponteiros

Em Git, **HEAD aponta para o commit atualmente selecionado. Normalmente, HEAD aponta para uma branch, e a branch aponta para um commit.** Quando você faz novo commit, o ponteiro da branch avança. Em prova, isso aparece indiretamente em comandos como reset, checkout, switch e detached HEAD. Não é necessário dominar internals do Git profundamente, mas é útil entender que commit, branch e HEAD não são sinônimos.

Conceito	Definição objetiva	Pegadinha
HEAD	Referência para o commit atualmente selecionado.	Não é necessariamente o último commit remoto.
Branch	Nome que aponta para uma linha de commits.	Não é uma cópia física completa do projeto.
Commit	Objeto histórico com snapshot, autor, mensagem e referência a pai(s).	Não é publicação remota no Git.
Remote tracking branch	Referência local que acompanha o estado conhecido de uma branch remota.	origin/main não é a mesma coisa que main local.

8.2 Detached HEAD

Detached HEAD ocorre quando HEAD aponta diretamente para um commit, e não para uma branch. **Isso pode acontecer ao fazer checkout de um commit antigo ou de uma tag.** O candidato deve saber a consequência geral: commits feitos nesse estado podem ficar sem nome de branch apontando para eles, se o desenvolvedor não criar uma branch ou referência. Em prova, a banca pode dizer que checkout de tag cria automaticamente uma nova branch permanente; isso é falso.

8.3 Conflito de merge: sequência mental

Conflito não é falha do Git nem do SVN: é uma indicação de que alterações concorrentes não puderam ser combinadas automaticamente. A ferramenta marca regiões conflitantes; o desenvolvedor decide o conteúdo final; depois marca a resolução e conclui a operação. Em Git, após resolver conflito de merge, normalmente usa-se git add nos arquivos resolvidos e finaliza-se o merge com commit, quando aplicável.

Fluxo conceitual de conflito em Git

```
git pull origin main          # busca e tenta integrar
# conflito em src/config.ts
# editar o arquivo, escolher conteúdo final e remover marcadores <<<<<< ===== >>>>>>
git add src/config.ts        # marca como resolvido
git commit                   # conclui merge, se necessário
```

8.4 Marcadores de conflito

Os marcadores <<<<<<, ===== e >>>>>> não são sintaxe de Java, TypeScript ou configuração; são marcadores temporários inseridos pela ferramenta para delimitar versões conflitantes. Comitar esses marcadores por engano é falha de resolução de conflito. A FGV pode cobrar leitura de trecho de código contendo esses sinais.

8.5 git cherry-pick e git bisect

Alguns comandos aparecem menos, mas podem ser usados como distratores fortes. git cherry-pick aplica um commit específico de outra linha de desenvolvimento à branch atual. git bisect ajuda a localizar, por busca binária, o commit que introduziu um problema. git blame mostra autoria linha a linha. Esses comandos não substituem merge, rebase, commit ou push.

Comando	Associação correta	Erro comum em alternativa
git cherry-pick	Aplicar commit específico em outra branch.	Dizer que integra toda a branch como merge.
git bisect	Achar commit defeituoso por busca binária.	Dizer que publica release.
git blame	Identificar commit/autor de linhas.	Dizer que desfaz alterações.
git clean	Remover arquivos não rastreados.	Confundir com limpar histórico.
git gc	Manutenção/coleta interna de objetos.	Confundir com controle de acesso.

9. Fluxos de trabalho com Git: o que pode cair sem virar Aula 14

Fluxo de trabalho não é o foco profundo desta aula, mas a FGV pode contextualizar controle de versão com revisão de código, branch protegida e merge request. A regra de ouro: Git é a ferramenta de versionamento; [GitHub](#), [GitLab](#), [Bitbucket](#) e Azure DevOps são plataformas que hospedam repositórios e adicionam recursos como pull request, merge request, issue, wiki, pipeline e permissões.

Fluxo	Ideia	Como a FGV pode cobrar
Feature branch	Cada funcionalidade ou correção é desenvolvida em branch própria e integrada após revisão.	Perguntar por isolamento de mudança e colaboração.
Git Flow	Modelo com branches como main/master, develop, feature, release e hotfix.	Cuidado com alternativa que trata como recurso nativo obrigatório do Git.
Trunk-based development	Integrações frequentes em tronco principal, com branches curtas e automação.	Pode aparecer ligado a CI/CD, mas CI/CD será aprofundado na Aula 14.
Pull request / merge request	Mecanismo de plataforma para revisão, discussão e integração de mudanças.	Não confundir com comando Git puro.
Branch protegida	Regra de plataforma que impede push direto ou exige aprovação/testes.	É governança de plataforma, não característica essencial do modelo distribuído.

10. Segurança, rastreabilidade e compliance no versionamento

Em ambientes públicos e bancários, versionamento serve também para rastreabilidade. Um commit com autor, data, mensagem e vínculo a demanda ajuda a reconstruir decisões. Mas versionamento mal utilizado pode expor segredo: token, senha, chave privada, certificado, dump de banco ou dado pessoal não deve ser commitado.

Pegadinha de prova: .gitignore não apaga histórico. Se um segredo foi commitado, adicioná-lo depois ao .gitignore apenas evita novos arquivos não rastreados com aquele padrão; não remove o que já foi versionado. Em situação real, é necessário revogar/rotacionar segredo e executar limpeza de histórico conforme política da organização.

Situação	Conduta correta
Senha apareceu em commit local ainda não publicado	Remover do commit antes do push; refazer commit se necessário.
Senha já foi enviada ao remoto	Rotacionar/revogar segredo e tratar limpeza de histórico conforme política.
Arquivo gerado aparece como untracked	Adicionar padrão ao .gitignore se não deve ser versionado.
Arquivo sensível já rastreado	.gitignore não basta; é preciso remover do índice/histórico conforme o caso.

11. Leitura de comandos: quadro de decisão rápida

Enunciado diz...	Pense em...
registrar alteração no histórico local	git commit
enviar alterações locais ao servidor remoto	git push
buscar alterações remotas sem aplicar na branch atual	git fetch
buscar e integrar alterações remotas	git pull
selecionar o que entrará no commit	git add / staging area
desfazer commit preservando histórico	git revert
reescrever histórico local	git reset / git rebase, conforme contexto
marcar release	git tag ou tags em SVN
criar working copy do SVN	svn checkout
atualizar working copy do SVN	svn update
publicar alteração no SVN	svn commit
bloquear arquivo binário	svn lock / file locking

12. Miniestudo de caso no estilo FGV

Imagine uma equipe do Poder Judiciário mantendo um módulo de consulta processual. O backend Quarkus expõe API REST, o frontend Angular consome os endpoints e as mudanças são revisadas por merge request. Um dev cria feature/consulta-avancada, altera ConsultaResource.java e consulta.service.ts, faz git add e git commit. Nesse momento, a alteração está registrada localmente. Para aparecer no repositório remoto, precisa de git push. Ao abrir merge request, a plataforma pode exigir aprovação e pipeline verde. Se main avançar, o dev pode git fetch para conhecer o novo estado remoto, git pull para integrar ou git rebase main para reaplicar seus commits sobre a base nova, conforme política do time. Se houver conflito, ele resolve arquivo a arquivo; a ferramenta não decide a

regra de negócio.

A mesma narrativa em SVN muda pontos essenciais: o dev teria uma working copy obtida por svn checkout, traria alterações por svn update e publicaria por svn commit diretamente no repositório central. Branches e tags seriam normalmente diretórios/cópias no repositório, sob convenções como trunk/branches/tags.

13. Pegadinhas FGV de alto ROI

Commit x push

Em Git, commit é local; push publica. Em SVN, commit já envia ao repositório central.

Fetch x pull

Fetch busca referências remotas; pull busca e integra.

SVN centralizado x Git distribuído

A diferença de arquitetura é a cobrança mais provável.

Branch x tag

Branch evolui; tag marca um ponto, normalmente release.

Revert x reset

Revert cria novo commit que desfaz; reset move ponteiros e pode reescrever histórico.

Merge x rebase

Merge preserva topologia; rebase reaplica commits e tende a histórico linear.

File locking

SVN tem suporte nativo típico; Git comum não tem file locking nativo como característica central.

GitHub/GitLab x Git

Git é o sistema de versionamento; GitHub/GitLab são plataformas que hospedam repositórios e adicionam colaboração.

Checkout em SVN x clone em Git

svn checkout cria working copy; git clone cria repositório local com histórico.

Conflito

Ferramenta detecta conflito, mas não entende regra de negócio; resolução é responsabilidade humana.

14. Questões autorais no padrão FGV

Resolva as questões sem consultar o gabarito. Os enunciados foram construídos para simular a cobrança da FGV: alternativas próximas, comandos com efeitos parecidos, cenários institucionais e distinções entre Git e SVN.

Questão 1. Em uma equipe responsável por sistemas internos de um tribunal, dois analistas discutem a adoção de Git ou Subversion (SVN) para um projeto legado que contém arquivos binários de layout, documentação e código-fonte. Considerando a arquitetura típica dessas ferramentas e a capacidade nativa de bloqueio de arquivos, assinale a afirmativa correta.

- A) Git e SVN são distribuídos, mas apenas Git possui file locking nativo.
- B) Git e SVN são centralizados, mas apenas SVN permite branches.
- C) Git é distribuído; SVN é centralizado; SVN possui suporte típico a file locking nativo.
- D) SVN é distribuído; Git é centralizado; Git possui file locking nativo por padrão.
- E) Git e SVN são equivalentes quanto à arquitetura, diferindo apenas nos nomes dos comandos.

Questão 2. Um desenvolvedor em uma branch feature/cadastro executou os comandos abaixo.

```
git add src/main/java/br/jus/Servico.java
git commit -m "Ajusta regra de cadastro"
```

No contexto do Git, é correto afirmar que, após esses comandos, a alteração:

- A) foi registrada no repositório local, mas ainda não foi necessariamente publicada no repositório remoto.
- B) foi publicada automaticamente no origin, pois todo commit Git é remoto.
- C) foi enviada ao servidor central, como ocorre no commit do SVN.
- D) foi apenas colocada na staging area, sem registro histórico.
- E) foi descartada da working tree e transferida para o stash.

Questão 3. Em relação aos comandos Git usados para sincronização com repositórios remotos, analise as afirmativas a seguir.

- I. git fetch obtém informações do repositório remoto, mas não integra automaticamente essas alterações à branch de trabalho atual.
- II. git pull, em uma configuração usual, equivale a buscar alterações remotas e integrá-las à branch atual.
- III. git push envia commits locais para um repositório remoto, quando há permissão e compatibilidade de histórico.

Está correto o que se afirma em:

- A) I, apenas.
- B) I e II, apenas.
- C) II e III, apenas.
- D) I, II e III.
- E) III, apenas.

Questão 4. Uma equipe mantém um monorepo com backend Java, frontend Angular e scripts SQL. Um analista deseja verificar quais arquivos foram modificados localmente e quais alterações já foram selecionadas para o próximo commit. O comando Git mais diretamente associado a essa verificação é:

- A) git log
- B) git status
- C) git push
- D) git clone
- E) git tag

Questão 5. Sobre SVN - Subversion, assinale V para a afirmativa verdadeira e F para a falsa.

- () A working copy é a área local em que o usuário edita arquivos versionados e interage com o repositório central.
- () O comando svn update é usado para trazer alterações do repositório para a working copy.
- () O comando svn commit registra alterações apenas no repositório local do usuário, exigindo posterior svn push.

As afirmativas são, respectivamente,

- A) V, V e F.
- B) V, F e V.
- C) F, V e V.
- D) F, F e V.

E) V, V e V.

Questão 6. Durante a correção de uma falha em produção, um desenvolvedor precisa desfazer o efeito de um commit já publicado em uma branch compartilhada, preservando a rastreabilidade do histórico. No Git, a alternativa mais adequada, em regra, é utilizar:

- A) git reset --hard diretamente na branch compartilhada.
- B) git revert, criando um novo commit de desfazimento.
- C) git checkout, pois checkout apaga commits publicados.
- D) git stash, pois stash desfaz commits remotos.
- E) git fetch, pois fetch reverte automaticamente alterações publicadas.

Questão 7. Considere o seguinte cenário em Git. Uma branch feature foi criada a partir de main. Enquanto a feature evoluiu, main recebeu novos commits. O desenvolvedor deseja reaplicar os commits da feature sobre o estado mais recente de main, produzindo histórico mais linear. O comando associado a essa ideia é:

- A) git merge main
- B) git push --force
- C) git rebase main
- D) git tag main
- E) git clone main

Questão 8. No modelo clássico de organização de um repositório SVN, relacione os diretórios às suas finalidades.

1. trunk
2. branches
3. tags

- () marcações de versões, normalmente tratadas como imutáveis por convenção.
- () linha principal de desenvolvimento.
- () linhas paralelas de desenvolvimento ou manutenção.

A sequência correta é:

- A) 3, 1, 2.
- B) 1, 2, 3.
- C) 2, 3, 1.
- D) 3, 2, 1.
- E) 1, 3, 2.

Questão 9. Assinale a afirmativa INCORRETA sobre controle de versão.

- A) Um sistema de controle de versão permite recuperar versões anteriores e auditar mudanças.
- B) Branches ajudam a desenvolver alterações em linhas paralelas.
- C) Conflitos podem exigir decisão humana, pois a ferramenta não conhece necessariamente a regra de negócio.
- D) Tags costumam ser usadas para marcar releases ou pontos relevantes do histórico.
- E) Controle de versão elimina a necessidade de testes, pois garante que todo código versionado esteja correto.

Questão 10. Uma equipe executou git fetch origin antes de revisar uma merge request. O efeito esperado desse comando é:

- A) enviar commits locais para origin.
- B) criar automaticamente um commit de merge na branch atual.
- C) obter referências e objetos do remoto, sem necessariamente alterar a branch de trabalho atual.
- D) descartar alterações locais não commitadas.
- E) converter o repositório distribuído em repositório centralizado.

Questão 11. Observe o trecho de comandos.

```
git switch -c feature/pagamento
echo "regra" >> regra.txt
git add regra.txt
git commit -m "Inclui regra de pagamento"
```

Após a execução bem-sucedida, considerando um repositório Git comum, assinale a afirmativa correta.

- A) A branch feature/pagamento foi criada e o commit foi registrado localmente nessa branch.
- B) O arquivo regra.txt foi enviado automaticamente ao repositório remoto.
- C) O comando git switch -c criou uma tag imutável chamada feature/pagamento.
- D) O git add publicou o arquivo no servidor central.
- E) O commit foi gravado no SVN remoto configurado como origin.

Questão 12. Em uma reunião de revisão, um analista afirma: "No Git, a staging area é desnecessária, pois todo arquivo modificado entra automaticamente no próximo commit". A afirmação é:

- A) correta, porque Git não diferencia arquivo modificado e arquivo selecionado.
- B) correta apenas quando existe repositório remoto.
- C) incorreta, pois a staging area seleciona o conjunto de alterações que será registrado no próximo commit.
- D) incorreta, pois staging area existe apenas no SVN.
- E) correta apenas para arquivos Java, mas não para arquivos TypeScript.

Questão 13. Quanto à diferença entre git merge e git rebase, assinale a alternativa correta.

- A) Merge sempre apaga a branch de origem; rebase sempre cria tag de release.
- B) Merge e rebase têm exatamente o mesmo efeito histórico e diferem apenas na mensagem de commit.
- C) Merge integra linhas de desenvolvimento preservando a topologia; rebase reaplica commits sobre outra base, podendo produzir histórico linear.
- D) Rebase é exclusivo para SVN; merge é exclusivo para Git.
- E) Merge reescreve necessariamente todos os commits públicos; rebase nunca reescreve histórico.

Questão 14. Uma equipe trabalha com arquivos de imagem e documentos binários que não são bem mesclados automaticamente. No contexto de controle de versão, a funcionalidade que ajuda a evitar edição concorrente desses arquivos é denominada:

- A) branching.
- B) file locking.
- C) rebasing.
- D) cherry-picking.
- E) stashing.

Questão 15. Analise as afirmativas a seguir sobre .gitignore.

- I. É usado para indicar padrões de arquivos que o Git deve ignorar quando ainda não estão versionados.
- II. É boa prática usá-lo para evitar versionar diretórios gerados, como target/ ou node_modules/.
- III. É o mecanismo adequado para remover do histórico senhas que já foram commitadas e enviadas ao remoto.

Está correto o que se afirma em:

- A) I e II, apenas.
- B) II, apenas.
- C) I e III, apenas.
- D) I, II e III.
- E) III, apenas.

Questão 16. No Git, um desenvolvedor quer guardar temporariamente alterações locais ainda não commitadas para trocar de branch e depois retomá-las. O comando mais associado a esse comportamento é:

- A) git tag
- B) git stash
- C) git blame
- D) git bisect
- E) git remote

Questão 17. Em uma auditoria técnica, deseja-se descobrir qual commit e qual autor introduziram determinada linha em um arquivo versionado no Git. O comando mais associado a essa investigação é:

- A) git blame
- B) git init
- C) git clean
- D) git remote add
- E) git gc

Questão 18. Sobre branch e tag, assinale a afirmativa correta.

- A) Branch e tag são sinônimos em Git e SVN.
- B) Tag é usada para evoluir desenvolvimento diário, enquanto branch é marcador fixo de release.
- C) Branch representa linha de desenvolvimento; tag marca um ponto específico do histórico, frequentemente uma versão.
- D) Branch só existe em SVN; tag só existe em Git.
- E) Tag sempre impede tecnicamente qualquer alteração futura no repositório.

Questão 19. Um servidor remoto chamado origin possui novos commits na branch main. O desenvolvedor local quer atualizar sua branch atual com esses commits em uma operação que busque e integre alterações, aceitando a possibilidade de conflito. O comando mais compatível com esse objetivo, em uso comum, é:

- A) git pull origin main
- B) git add origin main
- C) git status origin main
- D) git init origin main
- E) git tag origin main

Questão 20. Em um projeto público, a política de qualidade exige que mudanças sejam feitas em branches curtas, revisadas por pares e integradas à branch principal apenas após aprovação e testes automatizados. Em relação ao controle de versão, essa prática:

- A) dispensa o uso de commits, pois a aprovação substitui o histórico.
- B) é incompatível com Git, que não permite branches locais.
- C) combina controle de versão com colaboração e governança, mas não substitui testes nem revisão técnica.
- D) só é possível em SVN, pois Git não possui remotos.
- E) elimina conflitos de merge por definição.

15. Gabarito resumido

1-C, 2-A, 3-D, 4-B, 5-A, 6-B, 7-C, 8-A, 9-E, 10-C, 11-A, 12-C, 13-C, 14-B, 15-A, 16-B, 17-A, 18-C, 19-A, 20-C

16. Comentários completos

Questão 1 - Gabarito: C

Git é distribuído; SVN é centralizado; SVN possui suporte típico a file locking nativo. A alternativa A erra ao chamar SVN de distribuído e atribuir locking nativo ao Git. B erra ao centralizar Git e ao dizer que só SVN permite branches. D inverte as arquiteturas. E apaga a diferença central entre as ferramentas.

Questão 2 - Gabarito: A

No Git, git commit cria um registro no repositório local. Para publicar no remoto, usa-se git push. B e C confundem Git com modelo centralizado. D descreve o efeito de git add antes do commit. E descreve stash, não commit.

Questão 3 - Gabarito: D

As três afirmativas estão corretas: fetch busca dados remotos sem integrar automaticamente; pull faz fetch e integra; push envia commits locais ao remoto. As demais alternativas excluem itens verdadeiros.

Questão 4 - Gabarito: B

git status é o comando usado para ver arquivos modificados, staged e untracked. git log mostra histórico; push publica; clone copia repositório; tag cria marcador.

Questão 5 - Gabarito: A

SVN usa working copy local, update traz alterações do repositório e commit envia ao repositório central. O terceiro item é falso porque não existe ciclo svn commit local + svn push como no Git.

Questão 6 - Gabarito: B

git revert é adequado para desfazer mantendo histórico, pois cria novo commit inverso. reset --hard em branch compartilhada é perigoso e reescreve/desloca histórico. checkout não apaga commits publicados. stash guarda mudanças não commitadas. fetch não desfaz alterações.

Questão 7 - Gabarito: C

Rebase reaplica commits sobre outra base e tende a histórico linear. Merge integraria a branch sem necessariamente reescrever a base. push --force publica reescrita, não a realiza por si só. tag marca commit. clone copia repositório.

Questão 8 - Gabarito: A

tags marca versões; trunk é linha principal; branches são linhas paralelas. As outras sequências trocam finalidade de trunk, branches e tags.

Questão 9 - Gabarito: E

Controle de versão não garante correção funcional. Ele rastreia histórico, apoia colaboração e recuperação. A, B, C e D são afirmações corretas sobre VCS.

Questão 10 - Gabarito: C

git fetch baixa referências e objetos do remoto sem integrar automaticamente na branch atual. A é push. B descreve merge/pull. D seria restore/reset/clean dependendo do caso. E é absurdo conceitual.

Questão 11 - Gabarito: A

git switch -c cria e muda para nova branch; add seleciona arquivo; commit grava localmente. B e D confundem com publicação remota. C confunde branch com tag. E mistura Git com SVN.

Questão 12 - Gabarito: C

A staging area permite escolher o que entra no próximo commit. A é falsa porque Git distingue modified, staged e committed. B inventa dependência de remoto. D atribui staging ao SVN. E cria distinção por linguagem inexistente.

Questão 13 - Gabarito: C

Merge preserva topologia e pode criar commit de merge; rebase reaplica commits sobre nova base e tende a histórico linear. A, B, D e E trazem absolutismos ou exclusividades falsas.

Questão 14 - Gabarito: B

File locking bloqueia arquivo para evitar edição concorrente, muito útil em binários. Branching cria linhas paralelas; rebasing reaplica commits; cherry-picking aplica commit específico; stashing guarda mudanças temporárias.

Questão 15 - Gabarito: A

.gitignore ignora padrões de arquivos não versionados e é adequado para target/ e node_modules/. Ele não remove segredos já commitados do histórico; isso exige procedimentos de limpeza de histórico e rotação de credenciais.

Questão 16 - Gabarito: B

git stash guarda temporariamente alterações locais. tag cria marcador; blame identifica autoria de linhas; bisect ajuda a localizar commit problemático por busca binária; remote gerencia repositórios remotos.

Questão 17 - Gabarito: A

git blame associa linhas a commits e autores. init cria repositório; clean remove arquivos não versionados; remote add cadastra remoto; gc faz manutenção interna.

Questão 18 - Gabarito: C

Branch é linha de desenvolvimento; tag é marcador de ponto específico, como release. A e D negam conceitos básicos. B inverte funções. E exagera: tag deve ser tratada como imutável por governança, mas a impossibilidade técnica depende de proteção/configuração.

Questão 19 - Gabarito: A

git pull origin main busca e integra alterações de origin/main à branch atual, conforme uso comum. add seleciona arquivos; status mostra estado; init cria repositório; tag cria marcador.

Questão 20 - Gabarito: C

Branches curtas, revisão por pares e integração após testes combinam controle de versão com governança técnica, mas não substituem testes e revisão. A elimina commit indevidamente. B e D negam características do Git. E é falso: conflitos ainda podem ocorrer.

17. Checklist final de revisão

- VCS - Version Control System rastreia mudanças, autores, datas, justificativas e permite recuperação de versões.
- SVN - Subversion é centralizado; Git é distribuído.
- Em Git, commit é local; push publica; fetch busca sem integrar; pull busca e integra.
- Em SVN, commit envia ao repositório central; update traz mudanças para a working copy.
- SVN usa working copy e repositório central; Git usa working tree, index/staging area e repositório local.
- Merge integra histórias; rebase reaplica commits e pode reescrever histórico.
- Revert desfaz com novo commit; reset move ponteiros e pode descartar alterações conforme modo.
- Branch é linha de desenvolvimento; tag é marcador de release ou ponto do histórico.

- SVN tradicional usa trunk/branches/tags; tag no SVN é convenção de cópia tratada como imutável.
- File locking é associação forte com SVN; Git puro não deve ser marcado como ferramenta de locking nativo em questão conceitual.
- .gitignore evita versionar artefatos gerados e arquivos sensíveis ainda não versionados, mas não limpa segredo já commitado.
- Ferramentas de hospedagem como GitHub/GitLab não são o próprio Git; adicionam pull/merge request, revisão, issues, CI e permissões.

18. Fontes utilizadas

Edital retificado TJSC 2026 - Analista de Sistemas. Conteúdo programático: Engenharia de Software e Desenvolvimento, item 1.15 Ferramentas de controle de versão SVN e Git.

Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026, fonte fornecida no projeto.

FGV Conhecimento. Provas correlatas de tecnologia da informação com cobrança de Git, SVN, versionamento e arquitetura de sistemas, usadas apenas para calibrar estilo e densidade, sem cópia literal de questões.

Git documentation e Git Book. Referências para modelo distribuído, commits, branches, merge, rebase, fetch, pull e push.

GitHub Docs. Referência complementar sobre Git e Distributed Version Control System - DVCS.

Apache Subversion official documentation e Version Control with Subversion. Referências para working copy, repositório central, checkout, update, commit, trunk, branches e tags.